FIAP

# Guia Completo com Imagens

## Pipeline Olist: GitHub → Airflow → Snowflake

Data Integration e Pipelines — Aula 4

Prof. Rafael S Novo Pereira — FIAP MBA Data Engineering

Apache Airflow 3.1 • Astronomer Astro • Snowflake • Kubernetes

FIAP

## Sobre este guia

Este documento mostra o passo a passo com capturas de tela reais de como configurar e executar o pipeline completo da Aula 4. Ao final, você terá um pipeline orquestrado pelo Apache Airflow que baixa dados do GitHub, carrega no Snowflake, transforma em camadas Bronze/Silver/Gold e roda validações de qualidade.

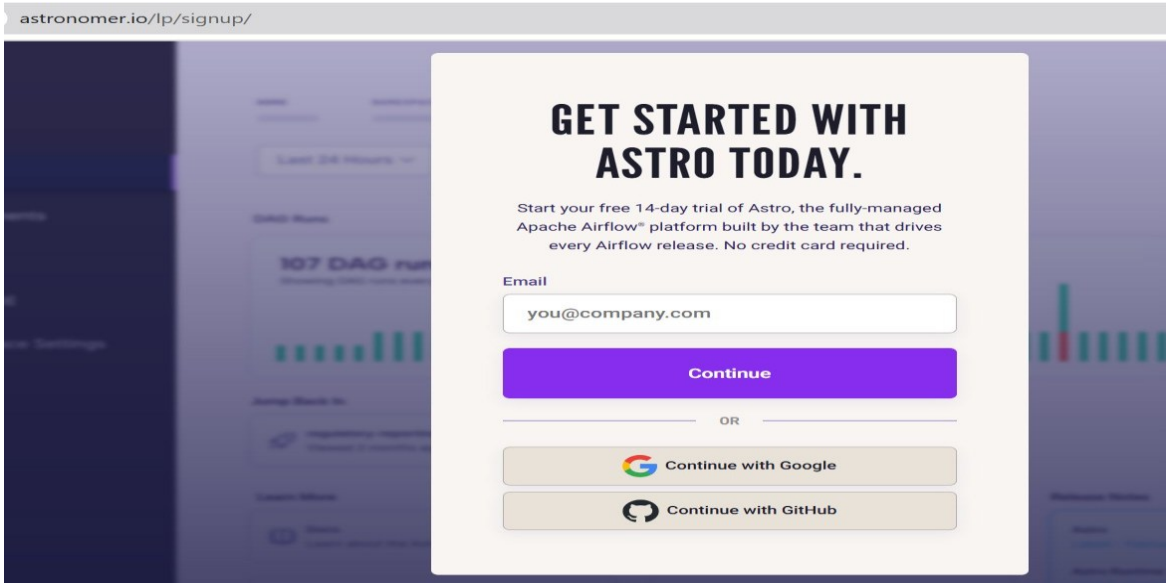
Pré-requisito: conta ativa no Snowflake (trial de 30 dias).

Tempo total estimado: 60-90 minutos.

---

## Etapa 1 — Criar conta no Astro

Acesse astronomer.io/lp/signup/ no navegador. Preencha seu email ou clique em "Continue with Google" ou "Continue with GitHub" para criar a conta.

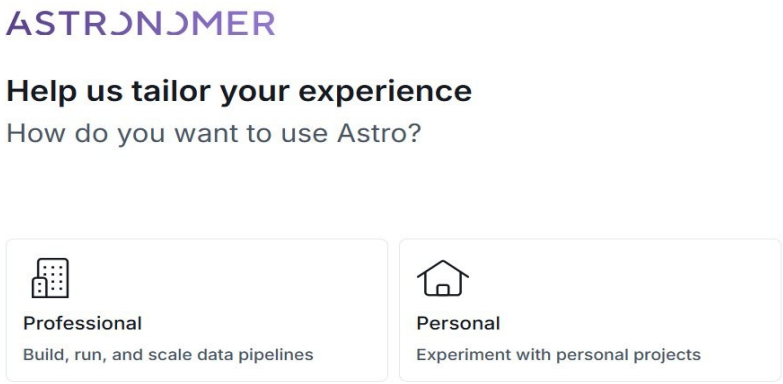



O trial é gratuito por 14 dias, sem cartão de crédito. Você recebe US$ 20 em créditos.

---

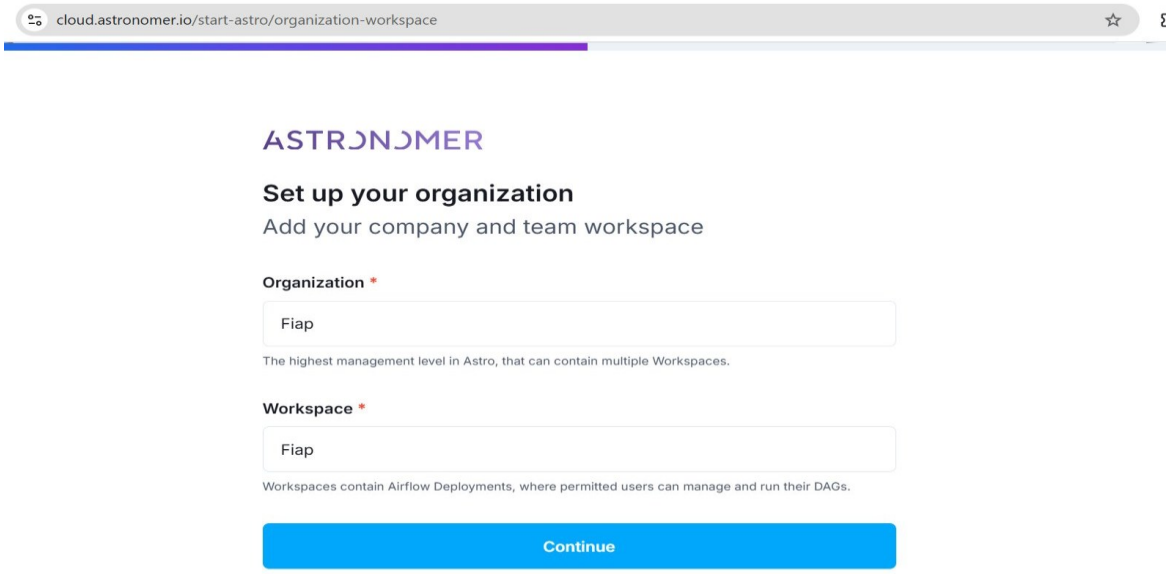
## Etapa 2 — Selecionar o uso

Na tela "Help us tailor your experience", selecione Professional.



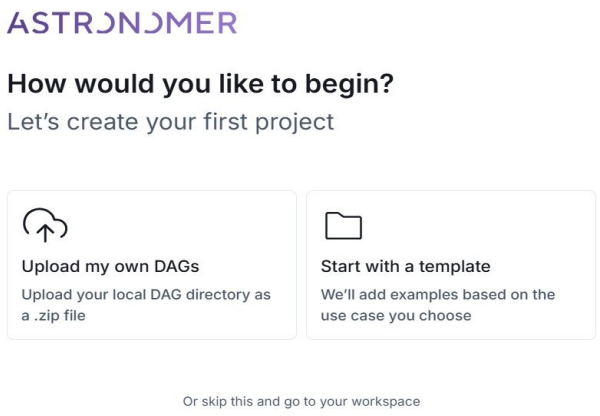

## Etapa 3 — Configurar Organization e Workspace

Preencha Organization como "Fiap" e Workspace como "Fiap" (ou o nome que preferir). Clique em Continue.



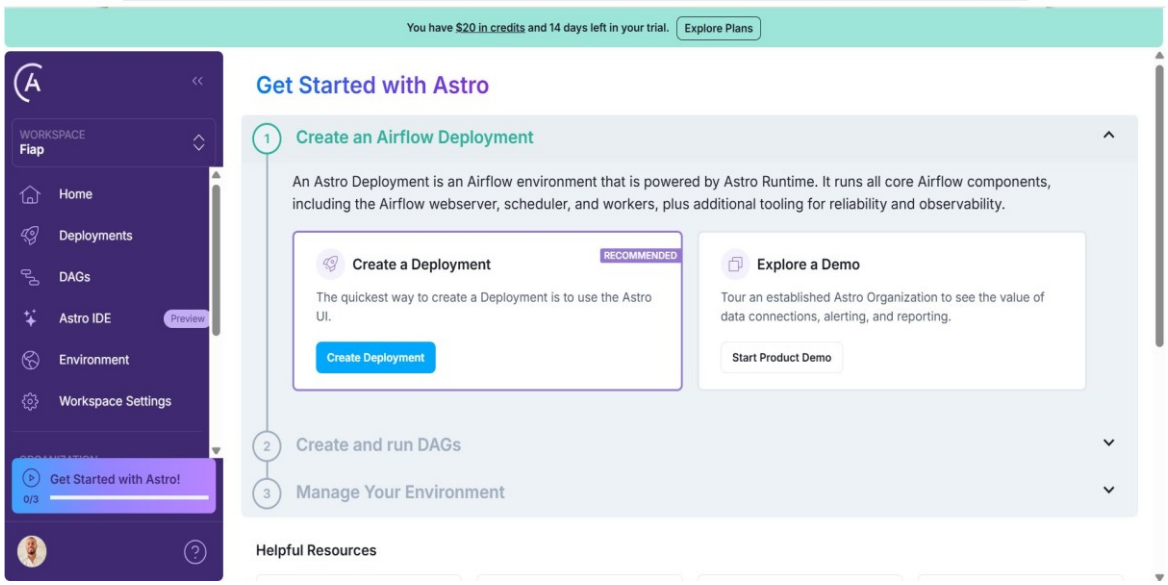
## Etapa 4 — Pular o template inicial

Na tela "How would you like to begin?", clique em "Or skip this and go to your workspace" no final da página. Vamos configurar o projeto manualmente.




## Etapa 5 — Tela principal do Astro

Você chega na tela principal. No passo 1, clique no botão verde "Create Deployment".

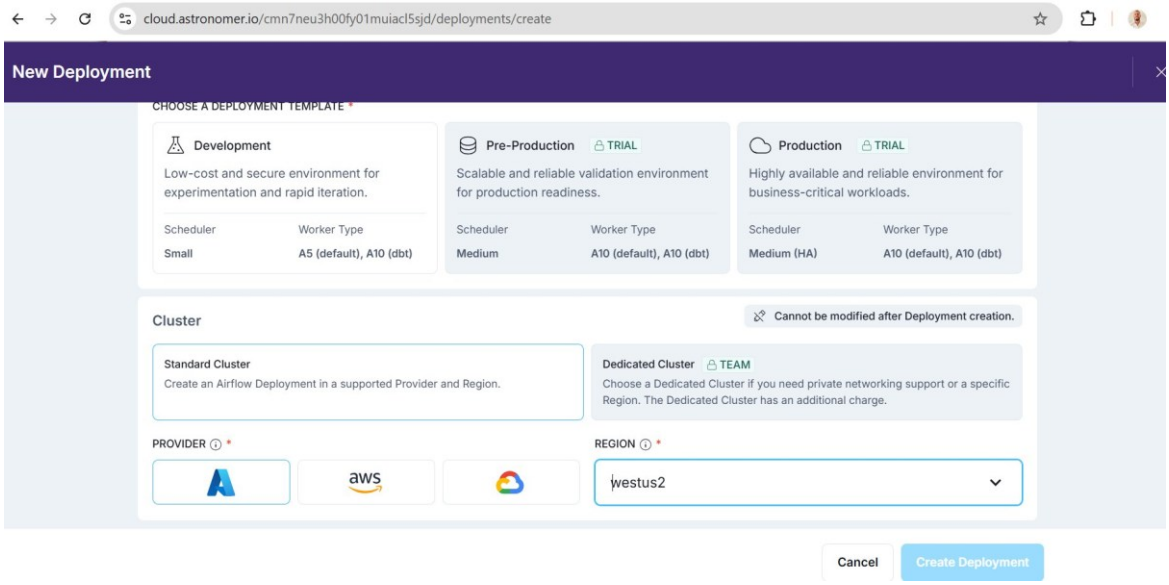



Observe no topo: "$20 in credits and 14 days left in your trial".

# Etapa 6 — Configurar o Deployment

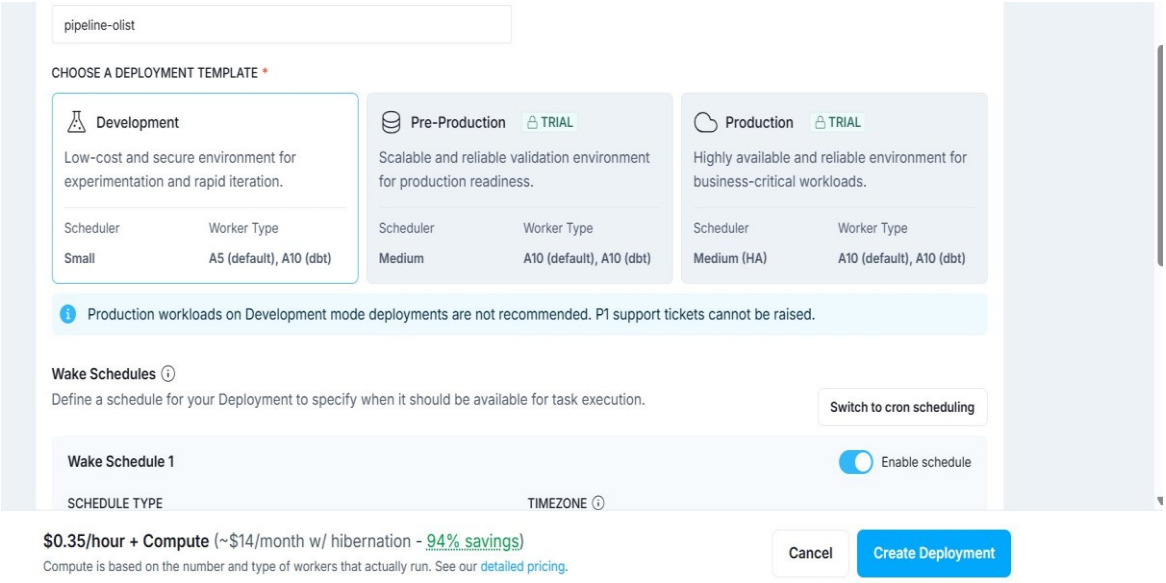
## 6.1 — Nome e template

Preencha o nome: pipeline-olist. Selecione o template Development (o mais barato). Provider: Astronomer. Region: westus2.



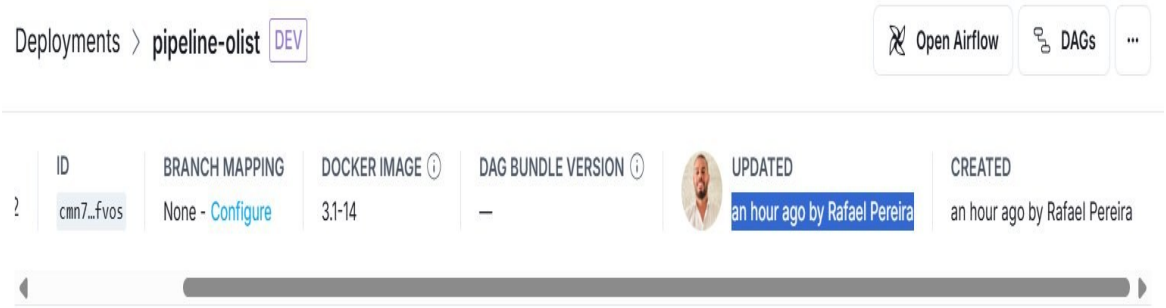

## 6.2 — Template Development selecionado

Confirme que Development está selecionado. Scheduler: Small, Worker Type: A5. Custo: $0.35/hora + Compute.



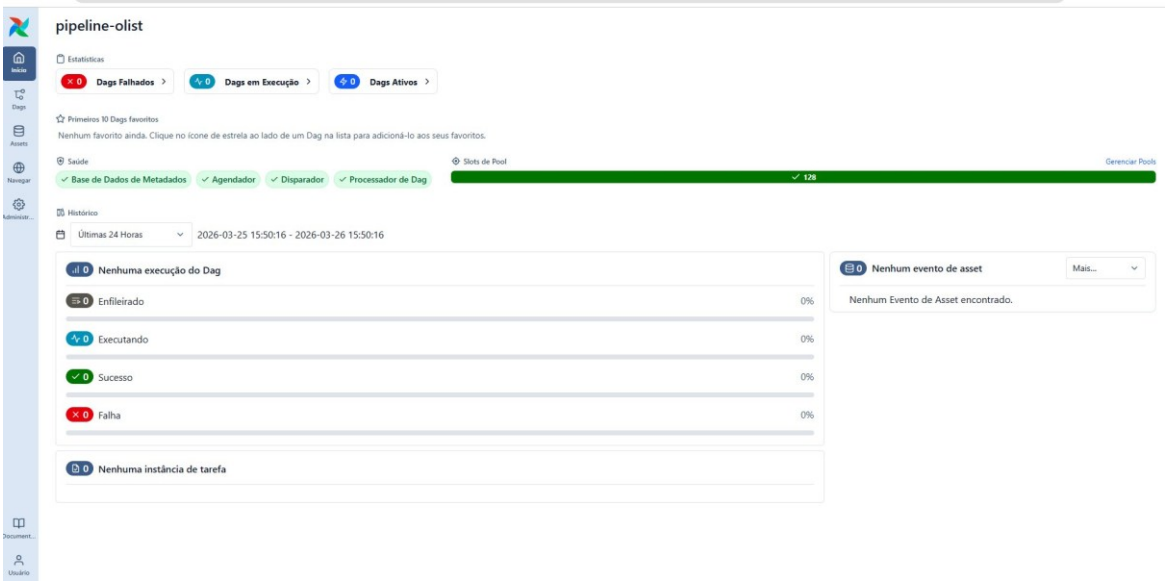
# Etapa 7 — Aguardar o Deployment ficar Healthy

Aguarde 2-5 minutos. O status vai mudar de "CREATING" para "HEALTHY". Quando aparecer o botão "Open Airflow" no canto superior direito, está pronto.




# Etapa 8 — Verificar o Airflow

Clique em "Open Airflow". A UI do Airflow vai abrir mostrando 4 componentes saudáveis: Base de Dados de Metadados, Agendador, Disparador e Processador de Dag. Todos verdes.



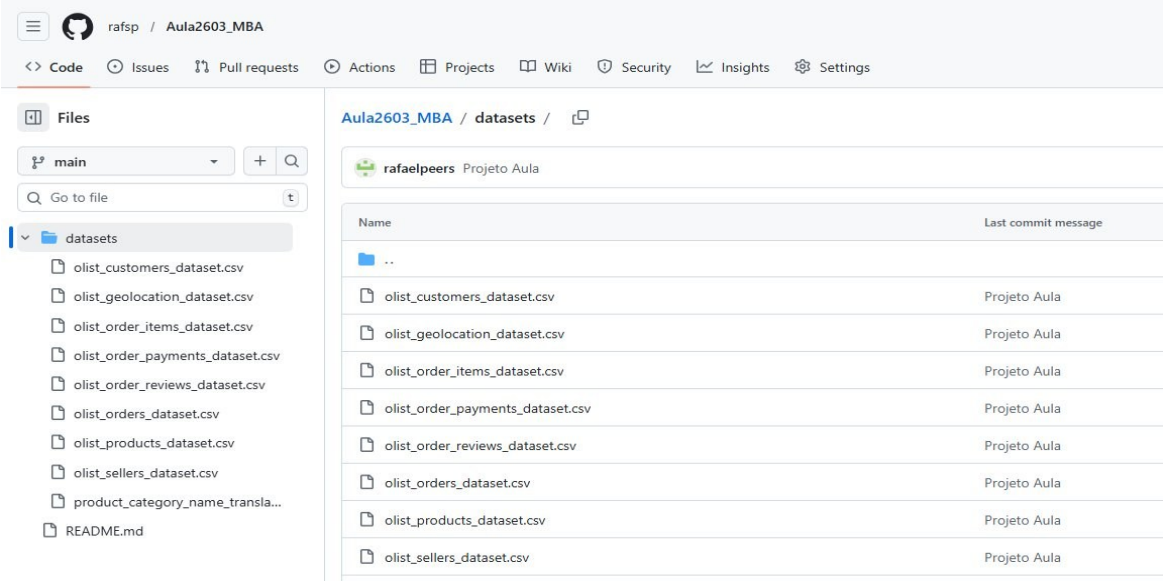

Neste ponto o Airflow está rodando, mas ainda não tem nenhum DAG. Vamos resolver isso nos próximos passos.

# Etapa 9 — Preparar o código no GitHub

O professor disponibilizou o código no repositório GitHub:

```
https://github.com/rafsp/Aula2603_MBA
```

O repositório contém: a pasta datasets/ com os CSVs do Olist, a pasta dags/ com o pipeline Python, o Dockerfile, requirements.txt e packages.txt.



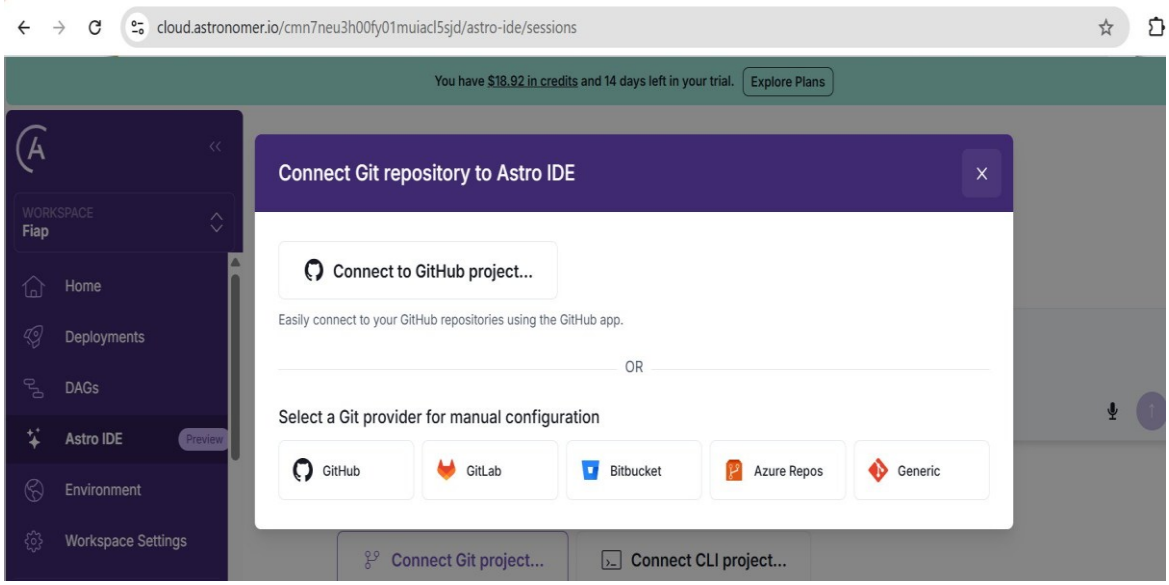

Os arquivos requirements.txt, Dockerfile e packages.txt na raiz do repositório são necessários para o Astro construir a imagem Docker com as dependências corretas (provider do Snowflake, pandas, etc.).

---

# Etapa 10 — Conectar o GitHub ao Astro IDE

## 10.1 — Abrir o Astro IDE

No Astro UI, clique em "Astro IDE" no menu lateral. Depois clique em "Connect Git project..." no meio da página.

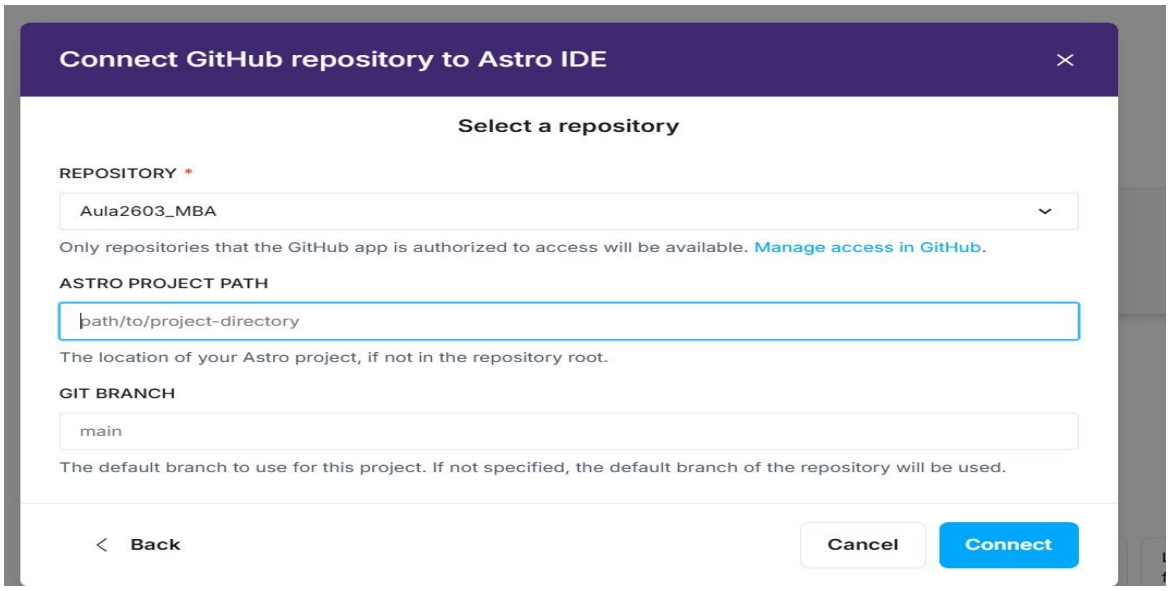


## 10.2 — Autorizar o GitHub

Clique em "Connect to GitHub project...". Autorize o acesso ao seu GitHub quando solicitado.

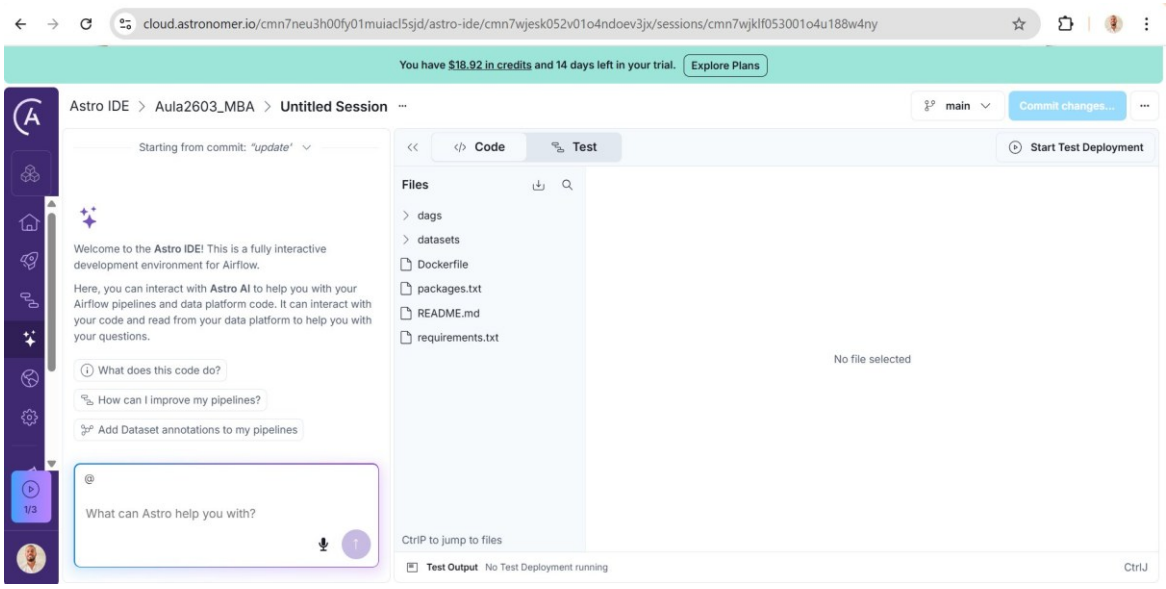
## 10.3 — Selecionar o repositório

Selecione o repositório Aula2603_MBA. Deixe o Astro Project Path vazio (o projeto está na raiz). Git Branch: main. Clique em Connect.




## 10.4 — IDE conectado

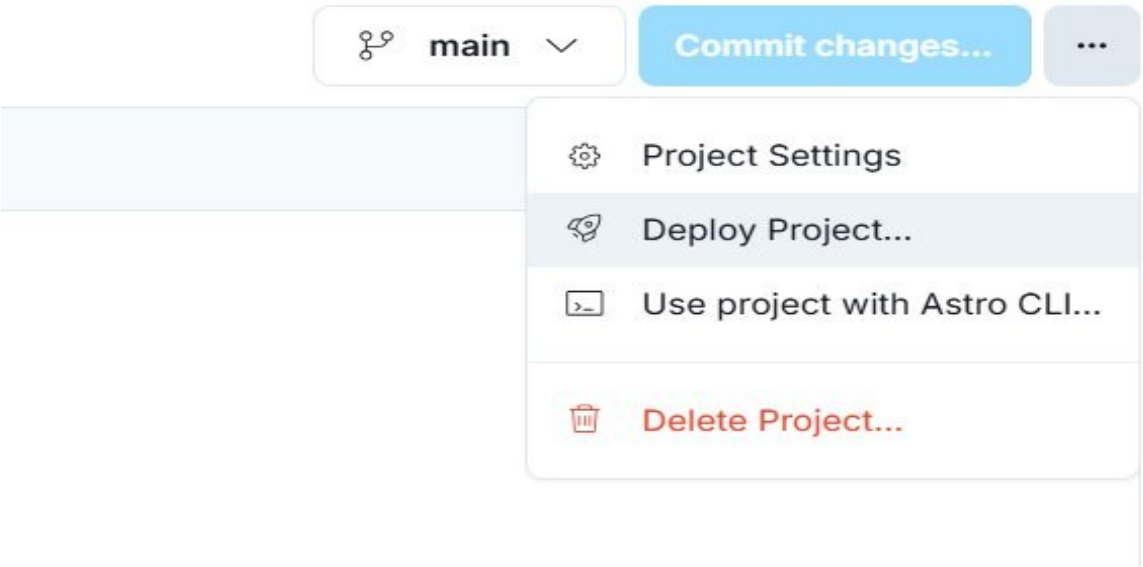
O Astro IDE abre mostrando os arquivos do repositório: dags, datasets, Dockerfile, packages.txt, README.md, requirements.txt.



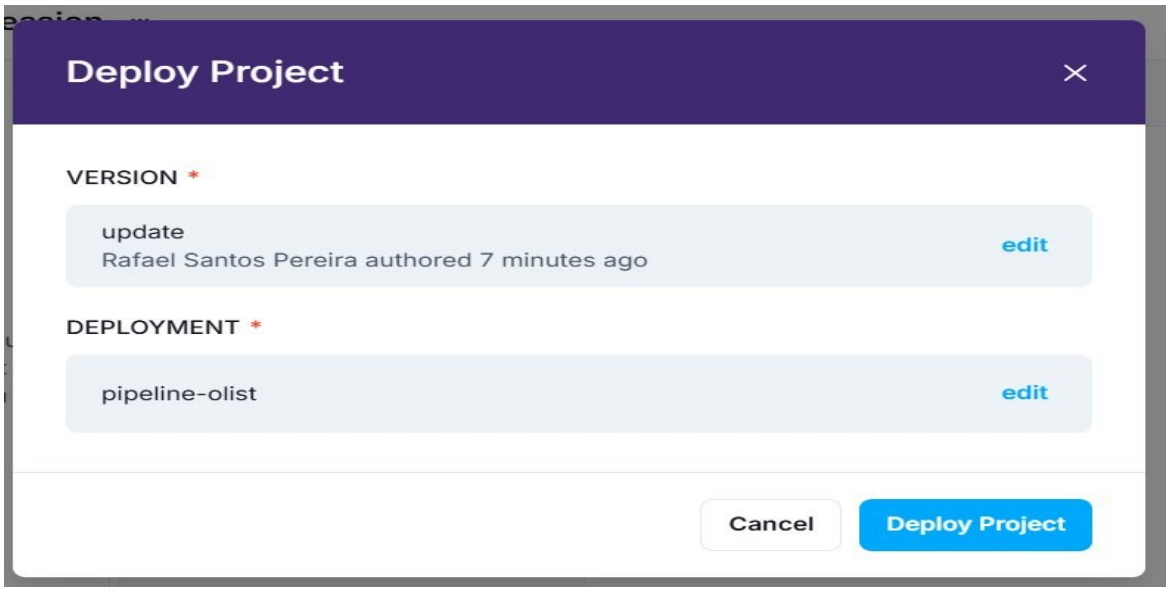
FIAP

## Etapa 11 — Deploy do projeto

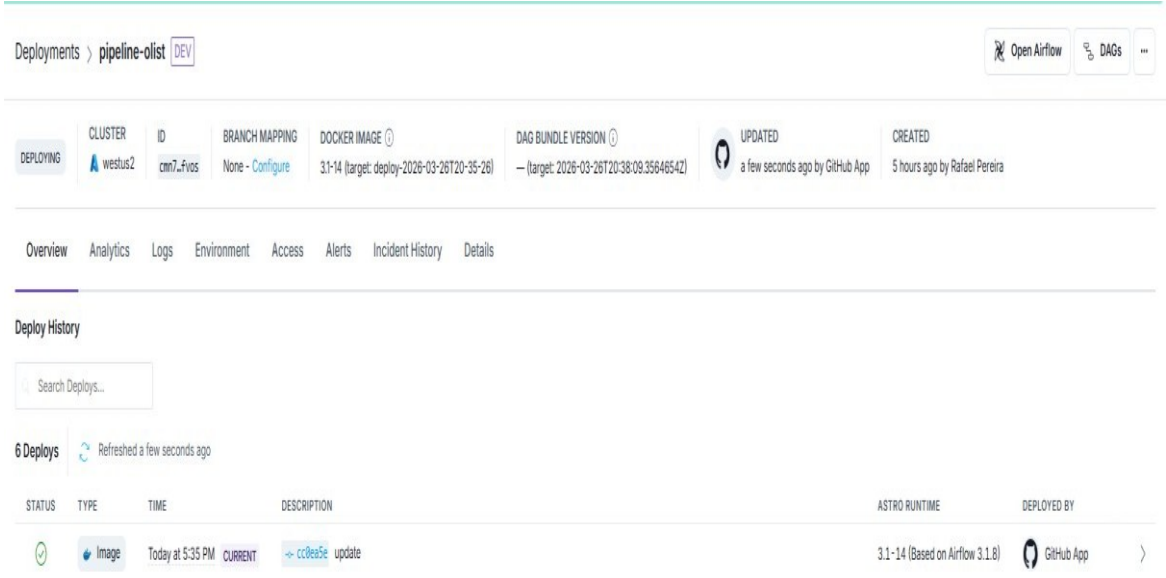
Clique nos três pontinhos "..." no canto superior direito do IDE. Selecione "Deploy Project...".




Na janela de confirmação, verifique que o Deployment é "pipeline-olist" e clique em "Deploy Project".

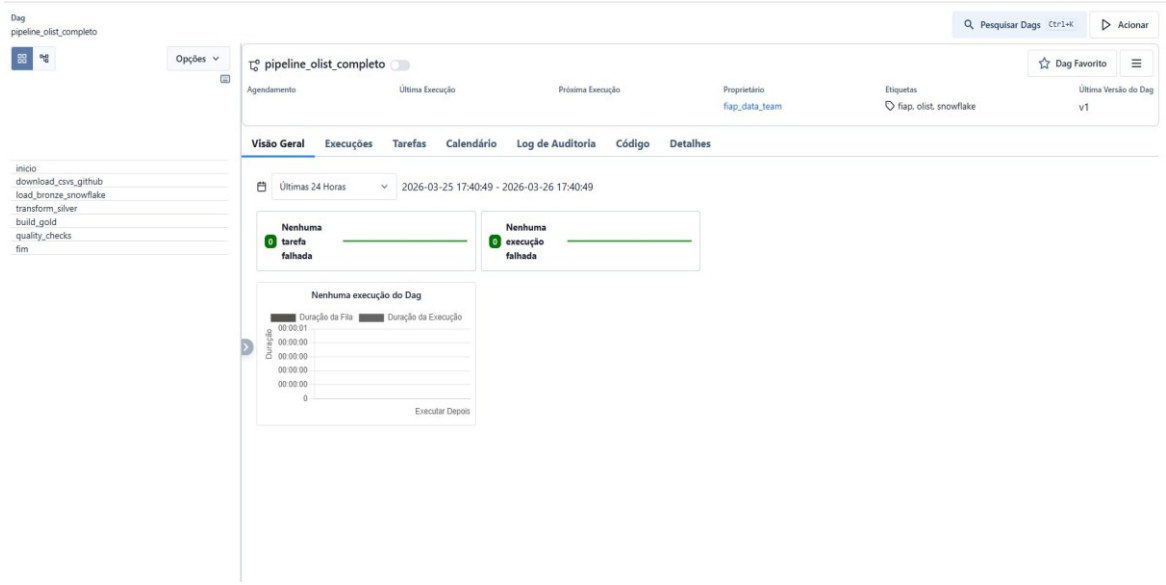



Aguarde 3-5 minutos. O Astro vai construir a imagem Docker com as dependências, instalar o provider do Snowflake e subir o DAG. Quando o status do Deployment voltar para HEALTHY, o deploy terminou.




## Etapa 12 — Verificar o DAG no Airflow

Clique em "Open Airflow". O DAG pipeline_olist_completo deve aparecer na lista. Você verá as tasks na lateral: inicio, download_and_load_bronze, transform_silver, build_gold, quality_checks, fim.

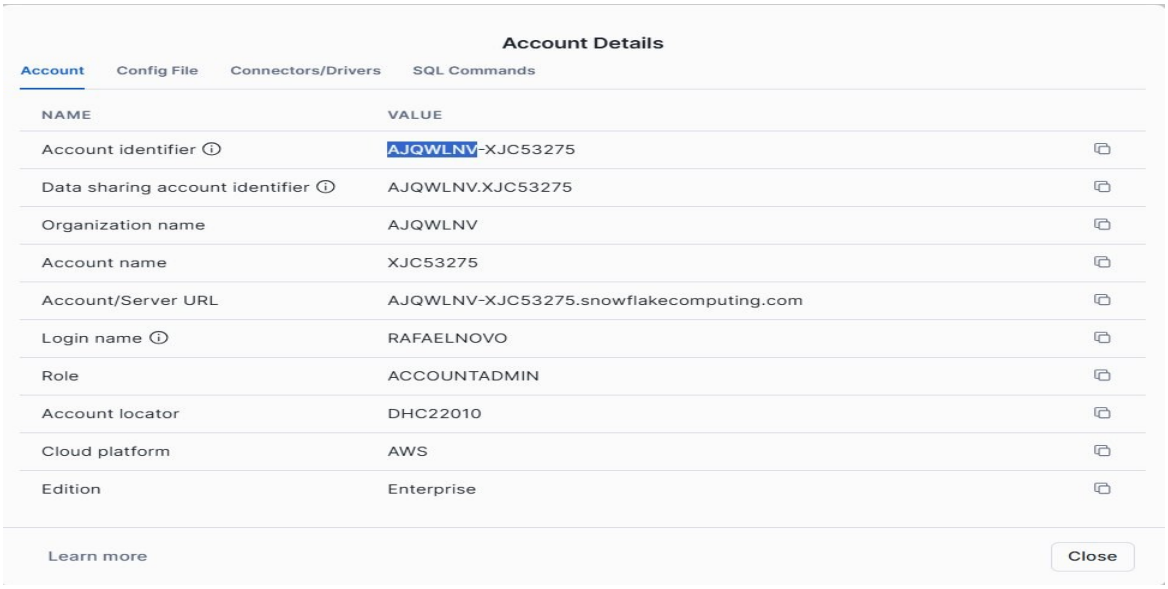


# Etapa 13 — Configurar a Connection com Snowflake

Esta é a etapa mais importante. Sem a Connection correta, as tasks que acessam o Snowflake vão falhar.

## 13.1 — Encontrar suas credenciais do Snowflake

Abra o Snowflake no browser. Clique no seu nome (canto inferior esquerdo) → Account Details. Anote o Account Identifier, Login name e Account/Server URL.




## 13.2 — Criar a Connection no Airflow

No Airflow, clique na engrenagem (Administr...) → Conexões → botão "+". Preencha:

ID da Conexão: snowflake_default

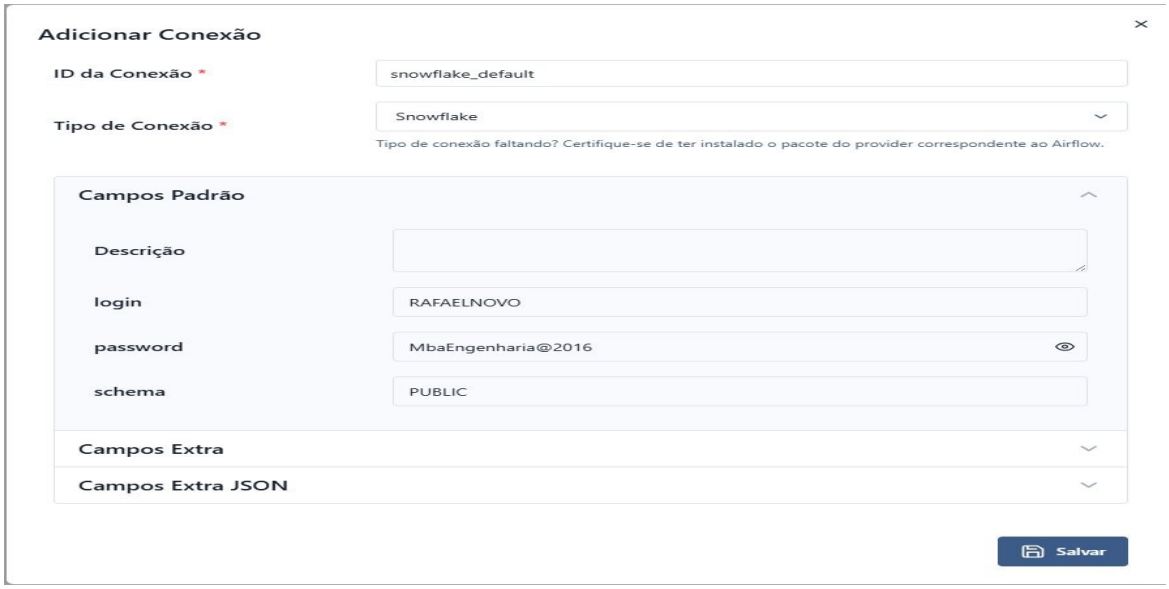
Tipo de Conexão: Snowflake

## 13.3 — Campos Padrão

login: seu Login name do Snowflake (ex: RAFAELNOVO)

password: sua senha do Snowflake

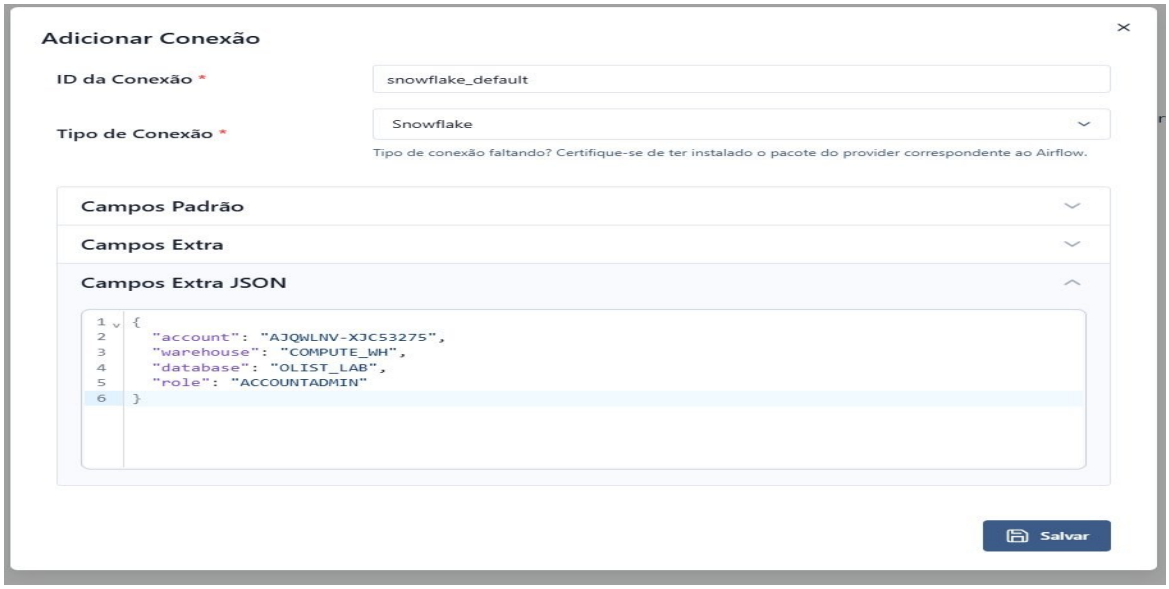
schema: PUBLIC




## 13.4 — Campos Extra JSON

Clique em "Campos Extra JSON" para expandir. Cole o JSON abaixo, substituindo o account pelo seu Account Identifier:

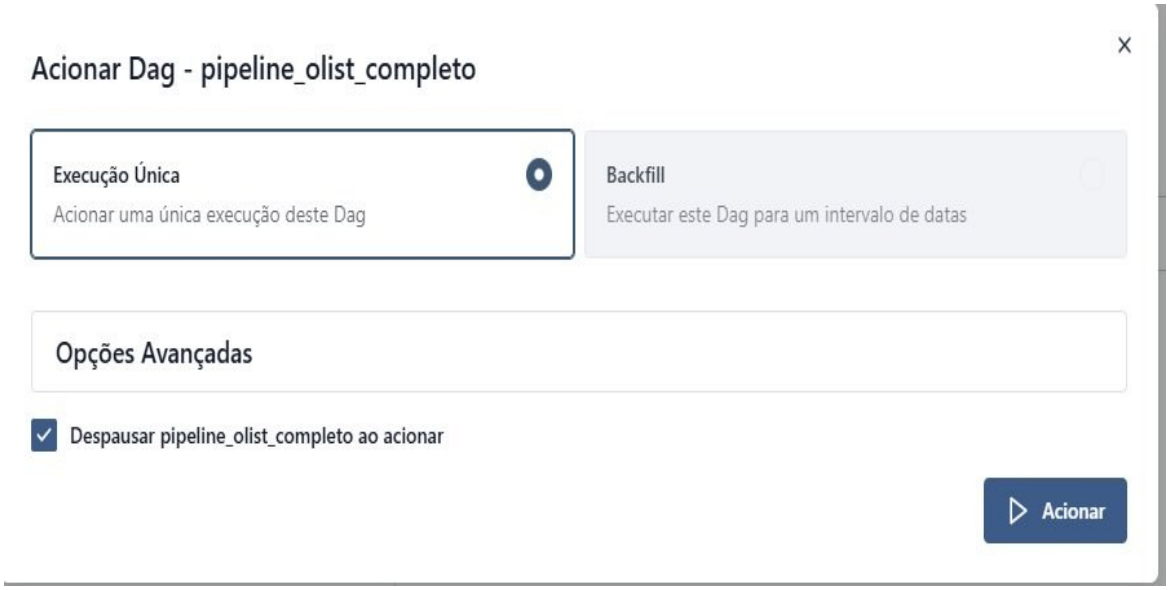
```
{"account": "SEU-ACCOUNT-IDENTIFIER", "warehouse": "COMPUTE_WH", "database":
"OLIST_LAB", "role": "ACCOUNTADMIN"}
```




Clique em Salvar.

## Etapa 14 — Executar o pipeline

Volte para a lista de DAGs. Clique no DAG pipeline_olist_completo. Clique em "Acionar" (botão play no canto superior direito). Selecione "Execução Única" e clique em "Acionar".

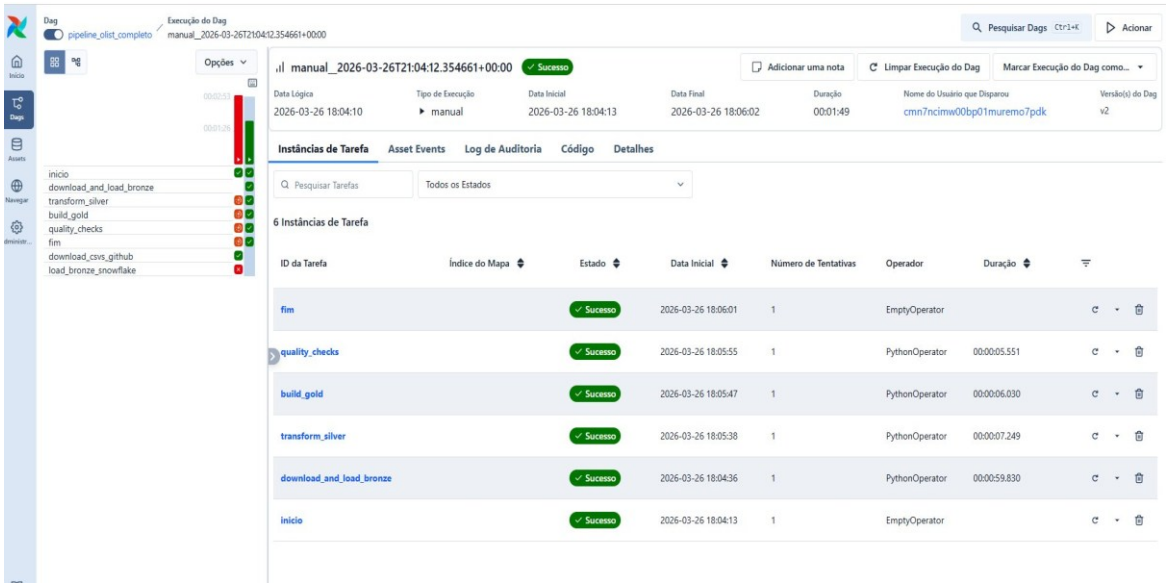



Acompanhe as tasks na lateral. Elas mudam de cor conforme executam. A execução completa leva aproximadamente 2 minutos.

---

## Etapa 15 — Pipeline completo com sucesso

Quando todas as tasks ficarem verdes, o pipeline terminou com sucesso. Veja a duração de cada task e o total.



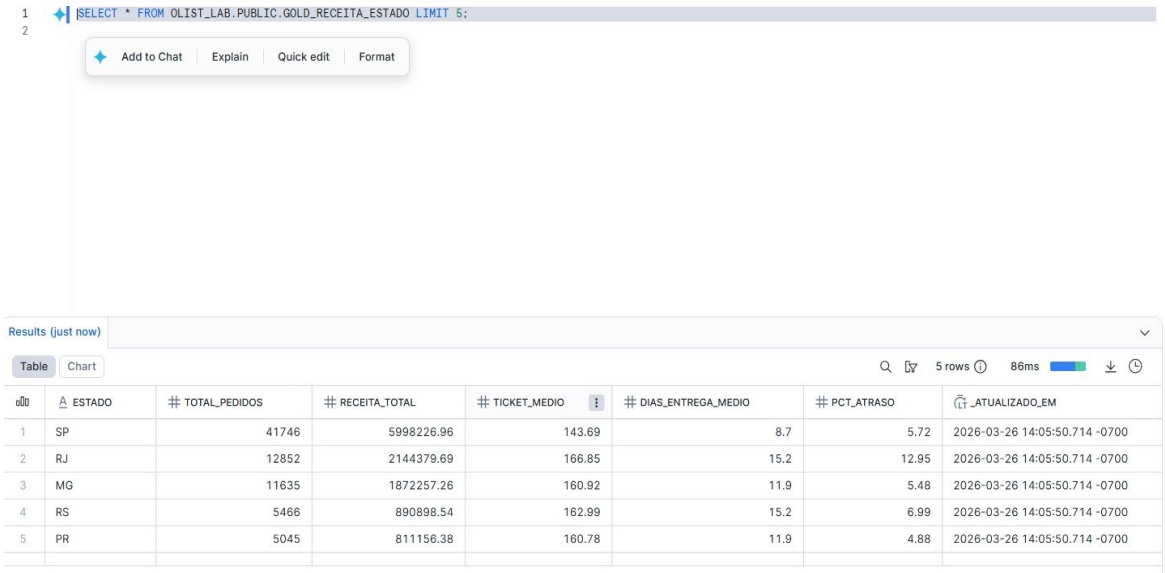

Observe: inicio → download_and_load_bronze (59s) → transform_silver (7s) → build_gold (6s) → quality_checks (5s) → fim. Total: ~1 minuto e 49 segundos.

---

# Etapa 16 — Verificar os resultados no Snowflake

Abra o Snowflake. Execute o SQL abaixo para ver os dados que o pipeline gerou:

```
SELECT * FROM OLIST_LAB.PUBLIC.GOLD_RECEITA_ESTADO LIMIT 5;
```




| | ESTADO | TOTAL_PEDIDOS | RECEITA_TOTAL | TICKET_MEDIO | DIAS_ENTREGA_MEDIO | PCT_ATRASO | _ATUALIZADO_EM |
|---|---|---|---|---|---|---|---|
| 1 | SP | 41746 | 5998226.96 | 143.69 | 8.7 | 5.72 | 2026-03-26 14:05:50.714 -0700 |
| 2 | RJ | 12852 | 2144379.69 | 166.85 | 15.2 | 12.95 | 2026-03-26 14:05:50.714 -0700 |
| 3 | MG | 11635 | 1872257.26 | 160.92 | 11.9 | 5.48 | 2026-03-26 14:05:50.714 -0700 |
| 4 | RS | 5466 | 890898.54 | 162.99 | 15.2 | 6.99 | 2026-03-26 14:05:50.714 -0700 |
| 5 | PR | 5045 | 811156.38 | 160.78 | 11.9 | 4.88 | 2026-03-26 14:05:50.714 -0700 |

SP lidera com R$ 6 milhões em 41.746 pedidos. RJ tem 12.95% de atraso nas entregas. Esses dados foram baixados do GitHub, carregados no Snowflake, transformados em Bronze/Silver/Gold e validados — tudo orquestrado pelo Airflow sem ninguém tocar manualmente no Snowflake.

---

# O que você acabou de fazer

1. Criou um ambiente Airflow gerenciado na nuvem (Astro) rodando em Kubernetes.
2. Conectou um repositório GitHub com o código do pipeline.
3. Fez deploy do código (Dockerfile + requirements.txt + DAG Python).
4. Configurou a conexão com o Snowflake (credenciais seguras no Airflow).
5. Executou um pipeline que baixou 7 CSVs, criou 7 tabelas Bronze, 1 tabela Silver com JOINs, 2 tabelas Gold com métricas de negócio, e rodou validações de qualidade.
6. Cada task rodou num pod Kubernetes isolado no Astro — você usou K8s sem perceber.

Na vida real, a única diferença é: a fonte seria uma API ou blob storage em vez de GitHub, o schedule seria diário em vez de manual, e o alerta iria para o Slack em vez de email. Mas o conceito é exatamente o mesmo.

---